



HTTP Request Smuggling

I. Khái niệm

HTTP request smuggling là một kỹ thuật tấn công nhằm lợi dụng cách các máy chủ xử lý chuỗi các yêu cầu HTTP. Khi các yêu cầu từ người dùng được gửi đến máy chủ phía trước (front-end) và được chuyển tiếp đến các máy chủ phía sau (back-end), sự khác biệt trong cách xử lý của các máy chủ này có thể dẫn đến lỗ hổng, cho phép kẻ tấn công thay đổi yêu cầu và gây ra những hậu quả nghiêm trọng như bỏ qua kiểm soát bảo mật, truy cập dữ liệu trái phép, hoặc tấn công người dùng khác.

II. CL.TE (Content-Length trên front-end, Transfer-Encoding trên back-end)

Trong tấn công CL.TE, máy chủ front-end xử lý yêu cầu dựa trên tiêu đề **Content-Length**, trong khi máy chủ back-end sử dụng **Transfer-Encoding** để quyết định vị trí kết thúc yêu cầu. Sự mâu thuẫn này dẫn đến việc máy chủ back-end không xử lý đầy đủ yêu cầu của máy chủ front-end, tạo ra cơ hội cho kẻ tấn công.

ví dụ:

```
POST / HTTP/1.1
Host: vulnerable-website.com
Content-Length: 13
Transfer-Encoding: chunked

0

SMUGGLED
```



- Máy chủ front-end đọc tiêu đề **Content-Length: 13**, xác định rằng yêu cầu có 13 byte dữ liệu (bao gồm "0" và "SMUGGLED"), và gửi yêu cầu này tới máy chủ back-end.
- Máy chủ back-end lại sử dụng **Transfer-Encoding: chunked** để phân tích yêu cầu. Khi thấy chunk đầu tiên có kích thước bằng 0, nó coi yêu cầu đã hoàn thành và bỏ qua phần "SMUGGLED", coi đó là phần đầu của yêu cầu tiếp theo.
- Phần "SMUGGLED" này được coi là yêu cầu mới, gây nhiễu yêu cầu tiếp theo của người dùng khác.

Kết quả:

Yêu cầu bị "đánh cắp" từ người dùng khác hoặc gây ra sự can thiệp trong luồng yêu cầu.

III. TE.CL (Transfer-Encoding trên front-end, Content-Length trên back-end)

Trong tấn công TE.CL, máy chủ front-end sử dụng tiêu đề **Transfer-Encoding** để quyết định nơi kết thúc yêu cầu, trong khi máy chủ back-end lại dùng **Content-Length**. Điều này dẫn đến việc back-end xử lý không chính xác yêu cầu, tạo ra không gian cho kẻ tấn công xen vào yêu cầu tiếp theo.

ví dụ:

```
POST / HTTP/1.1
Host: vulnerable-website.com
Transfer-Encoding: chunked
Content-Length: 3

8
SMUGGLED
0
```



- Máy chủ front-end dùng tiêu đề **Transfer-Encoding: chunked** và đọc chunk đầu tiên có độ dài 8 byte, bao gồm **"SMUGGLED"**, và gửi yêu cầu này tới máy chủ back-end.
- Máy chủ back-end lại sử dụng **Content-Length: 3**, xác định rằng yêu cầu có 3 byte dữ liệu, nhưng không nhận thấy chunk encoding. Phần **"SMUGGLED"** không được xử lý đúng cách và được coi là yêu cầu mới.

Kết quả:

Yêu cầu tiếp theo của người dùng khác bị ảnh hưởng bởi phần dữ liệu không mong muốn từ kẻ tấn công.

IV. TE.TE (Cả hai máy chủ sử dụng Transfer-Encoding, nhưng một máy chủ không xử lý đúng tiêu đề do bị làm mờ)

Trong dạng tấn công TE.TE, cả máy chủ front-end và back-end đều hỗ trợ **Transfer-Encoding**, nhưng một trong hai có thể bị làm mờ (obfuscation), dẫn đến việc không xử lý đúng tiêu đề. Kẻ tấn công sử dụng cách làm mờ tiêu đề **Transfer-Encoding** để tạo ra sự khác biệt giữa hai máy chủ.

ví dụ:

```
POST / HTTP/1.1
Host: vulnerable-website.com
Transfer-Encoding: xchunked

0

GET / HTTP/1.1
Host: victim-website.com
```



- Máy chủ front-end không nhận ra **Transfer-Encoding: xchunked** vì đó là một tiêu đề không chuẩn, và bỏ qua nó. Nó coi yêu cầu sử dụng **Content-Length** và gửi toàn bộ yêu cầu cho máy chủ back-end.
- Máy chủ back-end lại nhận ra tiêu đề **Transfer-Encoding** (hoặc không nhận ra sự khác biệt nhỏ trong tiêu đề) và phân tích yêu cầu dựa trên **chunked encoding**. Do đó, phần **"GET / HTTP/1.1"** được coi là yêu cầu tiếp theo.

Kết quả:

Kẻ tấn công có thể thêm yêu cầu của riêng mình (ở đây là **"GET / HTTP/1.1"**) vào luồng yêu cầu hợp lệ, gây ra nhiễu hoặc lấy cắp dữ liệu từ người dùng khác.

V. Các kỹ thuật phát hiện lỗ hổng HTTP Request Smuggling

Các lỗ hổng request smuggling có thể được phát hiện bằng cách sử dụng các kỹ thuật khác nhau, chủ yếu là dựa trên **thời gian phản hồi** và **phản hồi khác biệt**.

Kỹ thuật dựa trên thời gian phản hồi:

- Gửi các yêu cầu không đồng nhất giữa **Content-Length** và **Transfer-Encoding**, sau đó quan sát xem liệu có sự **chậm trễ** trong phản hồi hay không.

Kỹ thuật dựa trên phản hồi khác biệt:

- Gửi hai yêu cầu liên tiếp, trong đó yêu cầu đầu tiên được thiết kế để can thiệp vào yêu cầu thứ hai. Nếu phản hồi của yêu cầu thứ hai khác biệt (ví dụ: trả về lỗi 404 thay vì 200), lỗ hổng có khả năng tồn tại.

VI. Biện pháp phòng tránh

1. **Sử dụng HTTP/2 từ đầu đến cuối:** HTTP/2 có cơ chế mạnh mẽ để xác định độ dài của yêu cầu, giảm thiểu các lỗ hổng request smuggling.
2. **Loại bỏ việc hạ cấp HTTP/2:** Tránh chuyển đổi từ HTTP/2 sang HTTP/1 để tránh tạo ra sự khác biệt trong cách các yêu cầu được xử lý.



3. **Xử lý các yêu cầu không rõ ràng:** Máy chủ front-end nên chuẩn hóa các yêu cầu mơ hồ và máy chủ back-end cần loại bỏ các yêu cầu này để ngăn chặn các cuộc tấn công.
4. **Luôn xử lý thân yêu cầu:** Tránh giả định rằng yêu cầu sẽ không có thân (body), một trong những nguyên nhân gây ra lỗi hỏng CL.0 và desync từ phía client.
5. **Tắt lại sử dụng kết nối back-end:** Điều này giúp giảm thiểu một số loại tấn công liên quan đến request tunneling.

Các kỹ thuật kiểm tra và tấn công có thể được thực hiện qua việc gửi yêu cầu có cấu trúc đặc biệt, gây ra sự chậm trễ thời gian hoặc làm thay đổi phản hồi từ máy chủ. Một số biện pháp phòng tránh bao gồm sử dụng HTTP/2 từ đầu đến cuối, không hạ cấp HTTP/2 thành HTTP/1, và loại bỏ các kết nối khi gặp yêu cầu mơ hồ.

VII. Tham khảo

<https://portswigger.net/web-security/request-smuggling#what-is-http-request-smuggling>

Mọi thắc mắc hay ý kiến đóng góp vui lòng liên hệ :

Email : r3dctf.info@gmail.com

@Design by R3D CTF Team